

```

1  '''prmsailboat.py
2
3      This is the PRM skeleton code for the 2D sailboat problem.
4
5      PLEASE FINISH WRITING THE CODE, based on the 2D planner problem.
6      See also places marked by FIXME.
7
8      Use the PRM algorithm to find a path consistent with the wind.
9
10 '''
11
12 import bisect
13 import matplotlib.pyplot as plt
14 import numpy as np
15 import random
16 import time
17
18 from math          import inf, pi, sin, cos, sqrt, ceil, dist
19
20 from shapely.geometry import Point, LineString, Polygon, MultiPolygon
21 from shapely.prepared import prep
22
23 #####
24 #
25 #   Parameters
26 #
27 #   FIXME: Define the N/K...
28 #
29 N = 15
30 K = 2
31
32
33 #####
34 #
35 #   World Definitions (No Fixes Needed)
36 #
37 #   List of obstacles/objects as well as the start/goal.
38 #
39 (xmin, xmax) = (0, 10)
40 (ymin, ymax) = (0, 10)
41
42 # Collect all the obstacles and prepare (for faster checking).
43 obstacles = prep(MultiPolygon([

```

```

45     Polygon([[4,0], [4,4], [6,4], [6,0]]),
46     Polygon([[6,10], [8,8], [10,10]]),
47     Polygon([[0,6], [0,10], [4,10]],1))
48
49     # Define the start/goal states (x, y, theta)
50     (xstart, ystart) = ( 8, 4)
51     (xgoal, ygoal) = ( 2, 2)
52
53
54     #####
55     #
56     #   Visualization Class (No Fixes Needed)
57     #
58     #   This renders the world. In particular it provides the methods:
59     #       show(text = '')           Show the current figure
60     #       drawNode(node,           **kwargs) Draw a single node
61     #       drawEdge(node1, node2, **kwargs) Draw an edge between nodes
62     #       drawPath(path,           **kwargs) Draw a path (list of nodes)
63     #
64     class Visualization:
65         def __init__(self):
66             # Clear the current, or create a new figure.
67             plt.clf()
68
69             # Create a new axes, enable the grid, and set axis limits.
70             plt.axes()
71             plt.grid(True)
72             plt.gca().axis('on')
73             plt.gca().set_xlim(xmin, xmax)
74             plt.gca().set_ylim(ymin, ymax)
75             plt.gca().set_aspect('equal')
76
77             # Show the triangles.
78             for poly in obstacles.context.geoms:
79                 plt.plot(*poly.exterior.xy, 'k-', linewidth=2)
80
81             # Show immediately.
82             self.show()
83
84         def show(self, text = ''):
85             # Show the plot.
86             plt.pause(0.001)
87             # If text is specified, print and wait for confirmation.
88             if len(text)>0:
89                 input(text + ' (hit return to continue)')
90

```

```

91     def drawNode(self, node, **kwargs):
92         plt.plot(node.x, node.y, **kwargs)
93
94     def drawEdge(self, node, neighbor, **kwargs):
95         plt.arrow(node.x, node.y,
96                   (neighbor.x-node.x), (neighbor.y-node.y),
97                   head_width=0.2, head_length=0.1,
98                   length_includes_head=True, **kwargs)
99
100    def drawPath(self, path, **kwargs):
101        for i in range(len(path)-1):
102            self.drawEdge(path[i], path[i+1], **kwargs)
103
104
105    #####
106    #
107    #   A* Planning Algorithm - COPY FROM 2D PLANNER PROBLEM
108    #
109
110    def astar(nodes, start, goal):
111        # Clear the A* search tree information.
112        for node in nodes:
113            node.done      = False
114            node.seen      = False
115            node.parent    = None
116            node.creach     = 0
117            node.ctogoest  = inf
118
119        # Prepare the still empty *sorted* on-deck queue.
120        onDeck = []
121
122        # Begin with the start node on-deck.
123        start.done      = False
124        start.seen      = True
125        start.parent    = None
126        start.creach    = 0
127        start.ctogoest  = start.costToGoEst(goal)
128        bisect.insort(onDeck, start)
129
130        # Continually expand/build the search tree.
131        while True:
132            # Make sure we have something pending in the on-deck queue.
133            # Otherwise we were unable to find a path!
134            if not (len(onDeck) > 0):
135                return None
136

```

```

137         # Grab the next node (first on deck).
138         node = onDeck.pop(0)
139
140         # Mark this node as done and check if the goal is thereby done.
141         node.done = True
142         if goal.done:
143             break
144
145         # Add the neighbors to the on-deck queue (or update)
146         for neighbor in node.neighbors:
147             # Skip if already done.
148             if neighbor.done:
149                 continue
150
151             # Compute the cost to reach the neighbor via this new path.
152             creach = node.creach + node.costToConnect(neighbor)
153
154             # Just add to on-deck if not yet seen (in correct order).
155             if not neighbor.seen:
156                 neighbor.seen = True
157                 neighbor.parent = node
158                 neighbor.creach = creach
159                 neighbor.ctogoest = neighbor.costToGoEst(goal)
160                 bisect.insort(onDeck, neighbor)
161                 continue
162
163             # Skip if the previous path to reach (cost) was same or better!
164             if neighbor.creach <= creach:
165                 continue
166
167             # Update the neighbor's connection and resort the on-deck queue.
168             # Note the cost-to-go estimate does not change.
169             neighbor.parent = node
170             neighbor.creach = creach
171             onDeck.remove(neighbor)
172             bisect.insort(onDeck, neighbor)
173
174         # Build the path.
175         path = [goal]
176         while path[0].parent is not None:
177             path.insert(0, path[0].parent)
178
179         # Return the path.
180         return path
181
182     #

```

```

183 # Node Definition - COPY FROM 2D PLANNER PROBLEM
184 #
185
186 class Node():
187     #####
188     # Initialization:
189     def __init__(self, x, y):
190         # Define/remember the state/coordinates (x,y).
191         self.x = x
192         self.y = y
193
194         # Edges = set of neighbors. This needs to be filled in.
195         self.neighbors = set()
196
197         # Clear the status, connection, and costs for the A* search tree.
198         # TRUNK: done = True
199         # LEAF: done = False, seen = True
200         # AIR: done = False, seen = False
201         self.done = False
202         self.seen = False
203         self.parent = None
204         self.creach = 0 # Known/actual cost to get here
205         self.ctogoest = inf # Estimated cost to go from here
206
207     #####
208     # A* functions:
209     # Actual cost to connect to a neighbor and estimated to-go cost to
210     # a distant (goal) node.
211     def costToConnect(self, other):
212         return self.distance(other)
213
214     def costToGoEst(self, goal):
215         return self.distance(goal)
216
217     # Define the "less-than" to enable sorting in A*. Use total cost estimate.
218     def __lt__(self, other):
219         return (self.creach + self.ctogoest) < (other.creach + other.ctogoest)
220
221     #####
222     # PRM functions:
223     # Compute the relative distance to another node.
224     def distance(self, other):
225         return sqrt((self.x - other.x)**2 + (self.y - other.y)**2)
226
227     # Check whether in free space.
228     def inFreespace(self):

```

```

229         point = Point(self.x, self.y)
230         return obstacles.disjoint(point)
231
232
233     # Check the local planner - whether this connects to another node.
234     def connectsTo(self, other):
235
236         # Calculate distance using square root formula
237         distance = sqrt((other.x - self.x) ** 2 + (other.y - self.y) ** 2)
238
239         # Check how far left sailboat is
240         if (other.x - self.x) / distance < cos(120 * pi / 180):
241             return False
242
243         line = LineString([(self.x, self.y), (other.x, other.y)])
244         return obstacles.disjoint(line)
245
246     #####
247     # Utilities:
248     # In case we want to print the node.
249     def __repr__(self):
250         return "<Point %5.2f,%5.2f>" % (self.x, self.y)
251
252     # Compute/create an intermediate node. This can be useful if you
253     # need to check the local planner by testing intermediate nodes.
254     def intermediate(self, other, alpha):
255         return Node(self.x + alpha * (other.x - self.x),
256                     self.y + alpha * (other.y - self.y))
257
258     #
259     # PRM Functions - COPY FROM 2D PLANNER PROBLEM
260     #
261
262     def createNodes(N):
263         # Add nodes sampled uniformly across the space.
264         nodes = []
265         while len(nodes) < N:
266             node = Node(random.uniform(xmin, xmax),
267                         random.uniform(ymin, ymax))
268             if node.inFreespace():
269                 nodes.append(node)
270         return nodes
271
272     # Connect to up to K nearest neighbors (classic/standard approach)
273     def connectNearestNeighbors(nodes, K):
274         # Clear any existing neighbors, which are stored as a set.

```

```

275     for node in nodes:
276         node.neighbors = set()
277
278     # Check the neighbors.
279     for node in nodes:
280         # Examine the K nearest neighbors (ignoring node itself).
281         indicies = np.argsort(np.array([node.distance(n) for n in nodes]))
282
283         # Define total connections for each node and current neighbor
284         total_connect = 0
285         neighbor_index = 1
286
287         # Check if total connections has reached K neighbors for given node
288         while total_connect < K and neighbor_index < len(nodes):
289             neighbor = nodes[indicies[neighbor_index]]
290             # Force a undirected graph, so node is also neighbor's neighbor.
291             if (neighbor not in node.neighbors) and (node.connectsTo(neighbor)):
292                 node.neighbors.add(neighbor)
293                 # neighbor.neighbors.add(node)
294                 neighbor_index += 1          # Move to next neighbor in list
295
296     # Compute the path cost
297     def pathCost(path):
298         cost = 0
299         for i in range(1, len(path)):
300             cost += path[i-1].costToConnect(path[i])
301         return cost
302
303     # Post process the path
304     def postProcess(path):
305         # FIXME: This is just a placeholder. You should replace with
306         # actual processing.
307
308         i = 1
309         while i < (len(path) - 1):          # Don't include the start and goal nodes
310             predecessor = path[i - 1]
311             successor = path[i + 1]
312             if (predecessor.connectsTo(successor)):
313                 path.pop(i)                # Pop the node, but don't increment i
314                                           # since next node will fall into its place
315             else:
316                 i += 1
317
318     finalpath = path
319     return finalpath
320

```

```

321 #
322 # Main Code - COPY FROM 2D PLANNER PROBLEM
323 #
324
325 def main():
326     # Report the parameters.
327
328     print('Running with', N, 'nodes and', K, 'neighbors.')
329
330     # Create the figure. Some computers seem to need an additional show()?
331     visual = Visualization()
332     visual.show()
333
334     # Create the start/goal nodes.
335     startnode = Node(xstart, ystart)
336     goalnode = Node(xgoal, ygoal)
337
338     # Show the start/goal nodes.
339     visual.drawNode(startnode, color='orange', marker='o')
340     visual.drawNode(goalnode, color='purple', marker='o')
341     visual.show("Showing basic world")
342
343
344     # Create the list of nodes.
345     print("Sampling the nodes...")
346     tic = time.time()
347     nodes = createNodes(N)
348     toc = time.time()
349     print("Sampled the nodes in %fsec." % (toc-tic))
350
351     # Show the sample nodes.
352     for node in nodes:
353         visual.drawNode(node, color='k', marker='x')
354     visual.show("Showing the nodes")
355
356     # Add the start/goal nodes.
357     nodes.append(startnode)
358     nodes.append(goalnode)
359
360
361     # Connect to the nearest neighbors.
362     print("Connecting the nodes...")
363     tic = time.time()
364     connectNearestNeighbors(nodes, K)
365     toc = time.time()
366     print("Connected the nodes in %fsec." % (toc-tic))

```

```

367
368     # Show the neighbor connections.
369     for (i,node) in enumerate(nodes):
370         for neighbor in node.neighbors:
371             if neighbor not in nodes[:i]:
372                 visual.drawEdge(node, neighbor, color='g', linewidth=0.5)
373     visual.show("Showing the full graph")
374
375
376     # Run the A* planner.
377     print("Running A*...")
378     tic = time.time()
379     path = astar(nodes, startnode, goalnode)
380     toc = time.time()
381     print("Ran A* in %fsec." % (toc-tic))
382
383     # If unable to connect, show the part explored.
384     if not path:
385         print("UNABLE TO FIND A PATH")
386         # print(f"Number of connectsTo() calls: {connect_count}")
387         for node in nodes:
388             if node.done:
389                 visual.drawNode(node, color='r', marker='o')
390         visual.show("Showing DONE nodes")
391         return
392
393     # Show the path.
394     cost = pathCost(path)
395     visual.drawPath(path, color='r', linewidth=2)
396     visual.show("Showing the raw path (cost/length %.1f)" % cost)
397
398
399     # Post process the path.
400     finalpath = postProcess(path)
401
402     # Show the post-processed path.
403     cost = pathCost(finalpath)
404     visual.drawPath(finalpath, color='b', linewidth=2)
405     visual.show("Showing the post-processed path (cost/length %.1f)" % cost)
406     # print(f"Number of connectsTo() calls: {connect_count}")
407
408 if __name__ == "__main__":
409     main()

```